Vibe Coding 서버 라우팅 정리

FastAPI 기반 `server.py`의 엔드포인트를 기능별로 정리한 문서. 모든 `/api/*` 엔드포인트는 `Authorization: Bearer <token>` 헤더가 필요하며 (로그인/회원가입 제외), 토큰은 `SESSIONS` 딕셔너리(in-memory)에서 검증된다.

---

1. 엔드포인트 한눈에 보기

Method	Path	인증	역할
GET	`/`	✗	`chat_ui.html` 반환 (no-cache)
POST	`/api/login`	✗	로그인 → 토큰 발급
POST	`/api/logout`	✓	토큰 폐기
POST	`/api/register`	✗	회원가입
GET	`/api/chats`	✓	저장된 채팅/설정 복원
PUT	`/api/chats`	✓	채팅/설정 저장 + 감사 로그 기록
POST	`/api/chat`	✓	**자동 모드** — 명세→코드 한 번에 스트리밍
POST	`/api/chat/spec`	✓	**확인모드 1단계** — 명세 생성 또는 일반 채팅 응답
POST	`/api/chat/code`	✓	**확인모드 2단계** — 수정된 명세로 코드 스트리밍
GET	`/api/memory`	✓	사용자 영구 메모리 조회
PUT	`/api/memory`	✓	사용자 영구 메모리 저장
GET	`/api/specs`	✓	저장된 명세서 목록
GET	`/api/specs/{chat_id}`	✓	특정 명세서 조회
GET	`/{asset_path:path}`	✗	미리보기 iframe 정적 자산 fallback (204)

---

2. 인증 / 계정

`GET /` — UI 진입점

- server.py:117-122
- `chat_ui.html`을 `no-store` 캐시 헤더와 함께 반환.

`POST /api/login`

- server.py:130-136
- Body: `{username, password}`
- `users.json`의 PBKDF2(`sha256`, iters=200_000) 해시와 비교.

- 성공 시 `secrets.token_urlsafe(32)`로 생성한 토큰을 반환하고 `SESSIONS[token]=username`에 등록.

`POST /api/logout`

- server.py:139-144
- `Authorization` 헤더의 토큰을 `SESSIONS`에서 제거.

`POST /api/register`

- server.py:152-166
- 유효성: username ≥ 2자, password ≥ 4자, 중복 불가.
- 비밀번호는 `salt:hash` 형식으로 `users.json`에 append.

---

3. 채팅 기록 (복원용 + 감사용)

저장 구조:

```
chat_logs/
  <username>/
    state.json   # 프런트 복원용 (chats + settings)
    log.jsonl    # 턴 단위 감사 로그 {date,time,input,response}
```

`GET /api/chats`

- server.py:276-301
- 우선순위: `state.json` → (없으면) 구 포맷 `chat_logs/<user>.json` 폴백 → 빈 응답.

`PUT /api/chats`

- server.py:309-376
- `state.json`은 sanitize된 전체 덮어쓰기.
- `log.jsonl`은 **append-only**: 기존 (input, response) 쌍은 스킵, 새 턴만 추가 → 타임스탬프 보존.
- 구 단일 파일(`chat_logs/<user>.json`)이 있으면 조용히 삭제.

---

4. 채팅 파이프라인 — 두 가지 모드

모드 A: 자동 모드 `POST /api/chat`

- server.py:524-545
- 명세 생성과 코드 생성을 **하나의 스트림**으로 처리.
- `pipeline.pipe(...)`가 yield하는 청크를 `text/plain` StreamingResponse로 전달.

모드 B: 확인 후 진행 — 2단계

1단계 `POST /api/chat/spec`

- server.py:548-586
- `Pipeline._is_coding_request()` 판정:
  - **코딩 요청** → `{mode: "spec", spec: {...}}` (JSON, 비스트리밍)
  - **일반 질문** → `{mode: "chat", reply: "..."}` (스트림을 서버에서 합쳐서 반환)

2단계 `POST /api/chat/code`

- server.py:596-628
- 사용자가 검토/수정한 `spec`을 받아 `save_spec()`으로 갱신 후 코드 스트리밍.
- Body: `{message, chat_id, spec, messages}`.

공통: 요청 전처리 `_compose_effective_message`

- server.py:513-521
- 순서: **사용자 메모리 블록** → **PDF 첨부 텍스트** → **사용자 메시지**.
- 이미지는 `_parse_attachments`로 분리되어 `images` 파라미터로 파이프라인에 전달.
- PDF는 `pypdf`로 최대 20,000자까지 추출 (server.py:420-440).

---

5. 사용자 메모리 (영구 컨텍스트)

`user_memory/<username>.json` → `{"items": ["...", "..."]}` 매 채팅 요청마다 시스템 프롬프트에 주입됨. mtime 기반 캐시(`_MEMORY_CACHE`)로 디스크 I/O 최소화.

`GET /api/memory`

- server.py:495-498

`PUT /api/memory`

- server.py:505-510
- 최대 50개 항목, 공백 제거 후 저장.

---

6. 명세서 관리

`specs/<username>/*.json` 구조로 저장됨.

`GET /api/specs`

- server.py:637-661
- 사용자 소유 명세서 목록(파일명, chat_id, saved_at, user_message, project) 반환.

`GET /api/specs/{chat_id}`

- server.py:664-670
- 특정 chat_id의 명세 조회. `Pipeline.load_spec` 호출.

---

7. 미리보기 정적 자산 Fallback

`GET /{asset_path:path}`

- server.py:689-701
- 생성된 HTML 미리보기 iframe이 `style.css`, `script.js` 같은 외부 참조를 요청할 때 서버 로그에 404가 쌓이지 않도록 **204 No Content**로 응답.
- 허용 확장자: `.css` `.js` `.mjs` `.map` `.png` `.jpg` `.jpeg` `.gif` `.svg` `.webp` `.ico` `.woff` `.woff2` `.ttf` `.otf`
- 그 외 경로는 정상적으로 404 발생.
- ⚠️ catch-all이므로 **반드시 파일 맨 아래 배치**되어야 한다. 다른 라우트보다 먼저 등록하면 모든 요청을 가로채게 된다.

---

8. 요청 흐름 요약

```
[자동 모드]
  사용자 입력 → /api/chat
    → _compose_effective_message (메모리+PDF+메시지)
    → pipeline.pipe (명세→코드 스트림)
    → text/plain 스트림 응답

[확인 후 진행 모드]
  사용자 입력 → /api/chat/spec
    ├─ 코딩 요청? → spec JSON 반환 → [프런트에서 검토/수정]
    │                                → /api/chat/code → 코드 스트림
    └─ 일반 질문? → chat reply JSON 반환 (끝)
```

---

9. 인증 공통 헬퍼

- `_require_session(request)` (server.py:106-111): `Authorization: Bearer <token>` 파싱 후 `SESSIONS`에서 username 조회. 실패 시 401.
- `SESSIONS`는 in-memory 딕셔너리이므로 **서버 재시작 시 모든 토큰 무효화**.